Backtracking

这是一种通过探索所有可能的候选解来找出问题解的算法。如果候选解被确认不是一个有效的解（或者至少不是最后一个有效的解），回溯法会通过在上一步进行一些变化来丢弃该解，即“回溯”并尝试其他可能性。

```
void backtracking(参数) {
    if (终止条件) {
        存放结果;
        return;
    }
    // 在当前决策层，遍历所有可能的选项
    for (选择 : 本层集合中的元素) {
        处理节点;
        backtracking(路径, 选择列表); // 递归，基于当前的选择，进入下一层决策。
        撤销处理; // 回溯，当递归调用返回时（意味着下方的路要么走通了已保存结果，要么走不通失败了），需要恢复现场。
    }
}
```

- **试探性**：分步构建解决方案，每一步都尝试一个选项。
- **剪枝**：一旦发现当前路径无法得到正确解，立即放弃该路径（回溯），避免无效计算。
- **深度优先搜索 (DFS)**：通常基于深度优先搜索策略实现。

假设你在迷宫的一个岔路口（`backtracking` 函数被调用）：

1. 终止条件：如果你发现手里拿着钥匙且站在了出口（满足条件），大喊“我成功了！”（存放结果），然后转身往回走（return）。
2. For 循环：看着面前的三条路（左、中、右）。
3. 处理节点：先决定走左边这条路，迈出一脚（加入路径）。
4. 递归：沿着左边一直走到底（调用 `backtracking`）。
   - 如果在左边深处发现是死胡同，或者找到了出口，你会退回到刚才迈脚的地方。
5. 撤销处理：把刚才迈出的那只脚收回来（撤销路径），回到岔路口原点。
6. Next Iteration：现在 `for` 循环继续，你决定走中间这条路.....重复上述过程。

leetcode

46.全排列

- 给定一个不含重复数字的数组 `nums`，返回其 **所有可能的全排列**。

  输入： nums = [1,2,3]
  输出： [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

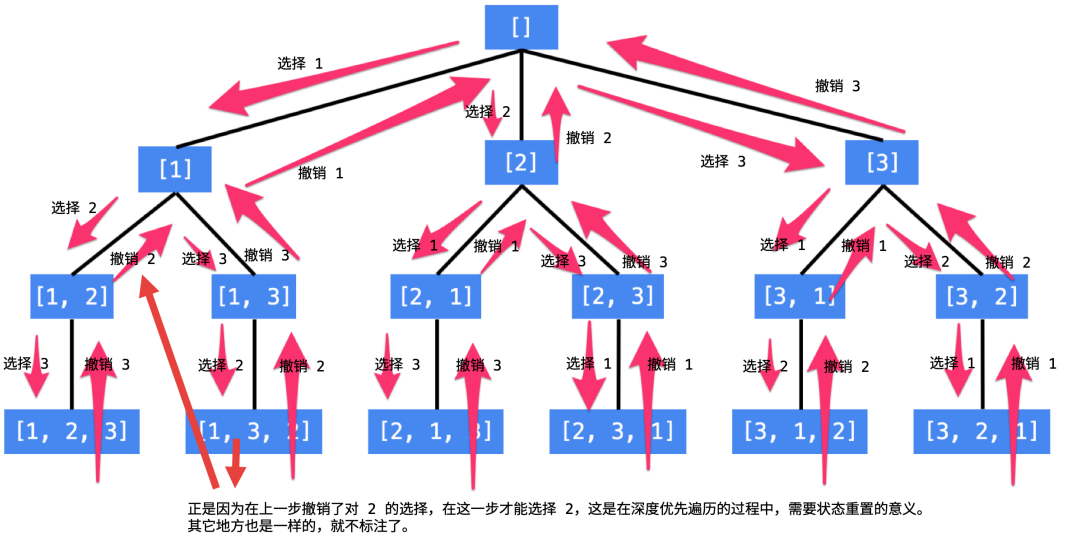
- 思路：全排列问题。

全排列问题可以看作是在构建一棵多叉树（决策树）：

- 根节点：空路径。
- 分支：每一步可以从剩余未使用的数字中选择一个。
- 叶子节点：当路径长度等于数组长度时，形成一个完整的排列。

回溯的本质就是：

1. 做选择：将某个数字加入当前路径。
2. 递归：基于这个选择，继续深入下一层。
3. 撤销选择：当递归返回（无论是因为找到了解还是无路可走），必须把刚才做的选择“undo”掉，恢复到进入这一层之前的状态，以便尝试下一个可能的选择。




```
class Solution {
public:
    void dfs(vector<int>& nums, vector<bool> &visited, vector<vector<int>>
&res, vector<int> &path){
        // 终止条件
        if( path.size()==nums.size() ){ // 数组元素遍历完了，构建了一个完整的序列
            res.push_back(path);
            return;
        }
        for( int i=0; i<nums.size(); i++ ){ // 选择本层集合中的元素
            if(visited[i]) continue; // 剪枝：如果该元素已被使用，跳过
            // 处理元素
            visited[i]=true; // 标记nums[i]已被占用
            path.push_back(nums[i]); // 将当前的“选择”添加到路径记录中
            // 递归处理
            dfs(nums,visited,res,path); // 当递归返回时（意味着以当前 nums[i] 开头
的所有排列都已经找完了），需要恢复现场。
            // 撤销处理
            path.pop_back();
            visited[i]=false;
        }
    }
```

```
    vector<vector<int>> permute(vector<int>& nums) {
        vector<vector<int>> res; // 存放全排列的结果
        vector<bool> visited(nums.size(),false); // 标记数组，记录哪些元素已经被使用过
        vector<int> path; // 存储当前解空间的路径，即当前正在构建的排列
        dfs(nums,visited,res,path);

        return res;
    }
};
```

-

78.子集

解决此类**组合/子集**问题的通用回溯模板

```
void backtrack(起始位置 index) {
    收集结果; // 子集问题特有

    for (i = index; i < 总数; i++) {
        做选择;
        backtrack(i + 1); // 关键点：下一层从 i+1 开始
        撤销选择;
    }
}
```

- 一个整数数组 `nums` ，数组中的元素 **互不相同** 。返回该数组所有可能的**子集**。

  输入： nums = [1,2,3]
  输出： [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

- 思路：

  我们可以把求子集的过程想象成遍历一棵多叉树：

  - 根节点：空集 `[]` 。
  - 分支：对于数组中的每个数字，我们都有"选"或"不选"的决策。但为了避免重复（如 `[1,2]` 和 `[2,1]` 是同一个子集），我们规定只能向后选。
  - 节点含义：树上的每一个节点（从根到叶子的路径上的任意一点）都代表一个合法的子集。

```
class Solution {
public:
    void dfs(vector<int>& nums, vector<vector<int>> &res, vector<int> &path,int index){
        // 子集问题不需要等到终止条件才收集，因为每一个节点都是一个合法的子集
        res.push_back(path);
        // 选择本层集合中的元素。index保证不重复选取，只能向后选取
        // 如果当前选了 nums[i]，下一层只能从 i+1 及其之后的元素中选，不能回头选前面的
        for( int i=index;i<nums.size();i++ ){ // 之前的元素（下标小于 index 的）已经被处理过
```

```
            // 处理元素
            path.push_back(nums[i]); // 将当前的“选择”添加到路径记录中
            dfs(nums,res,path,i+1); //
            // 撤销处理
            path.pop_back();
        }
        return;
    }

    vector<vector<int>> subsets(vector<int>& nums) {
        vector<vector<int>> res; // 存储子集的结果
        vector<int> path; // 存储当前的路径
        dfs(nums,res,path,0);

        return res;
    }
};
```

-

17.电话号码的字母组合

- 一个仅包含数字 `2-9` 的字符串，返回所有它能表示的字母组合。

```
输入: digits = "23"
输出: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

- 思路：

  我们可以把这个问题想象成走迷宫或者爬树：

  - 树的深度：由输入字符串 `digits` 的长度决定。例如输入 "23"，树就有 2 层（不包括根节点）。
  - 树的分支：每一层有多少个分支，取决于当前数字对应的字母个数。
    - 数字 '2' 对应 "abc"，所以第一层有 3 个分支。
    - 数字 '7' 对应 "pqrs"，所以某一层可能有 4 个分支。
  - 目标：找出从根节点到所有叶子节点的路径，每一条路径就是一个合法的字母组合。

```
class Solution {
    // 构建hash表
```

```
    unordered_map<char,string> digitsToLetters{
        {'2', "abc"},
        {'3', "def"},
        {'4', "ghi"},
        {'5', "jkl"},
        {'6', "mno"},
        {'7', "pqrs"},
        {'8', "tuv"},
        {'9', "wxyz"}
    };
public:
    void dfs(vector<string>& res,string& digits,string& path,int index){
        // 终止条件，长度相同，已经为每一个数字都选择了一个字母
        if(path.length() == digits.length()){
            res.push_back(path);
            return;
        }
        // 获取当前层的数字和对应的字母串
        char digit = digits[index];
        string letters = digitsToLetters[digit];
        // 遍历当前数字对应的所有字母
        for(int i = 0;i<letters.length();i++){
            // 处理元素
            path.push_back(letters[i]); // 将当前字母加入路径
            // 递归处理
            dfs(res,digits,path,index+1); // 处理下一个数字 (index + 1)
            // 撤销处理
            path.pop_back(); // 回溯，移除最后一个字母，尝试下一个字母
        }
    }
    vector<string> letterCombinations(string digits) {

        vector<string> res; // 存储字母组合结果
        string path; // 存储当前组合的结果
        dfs(res,digits,path,0);

        return res;
    }
};
```

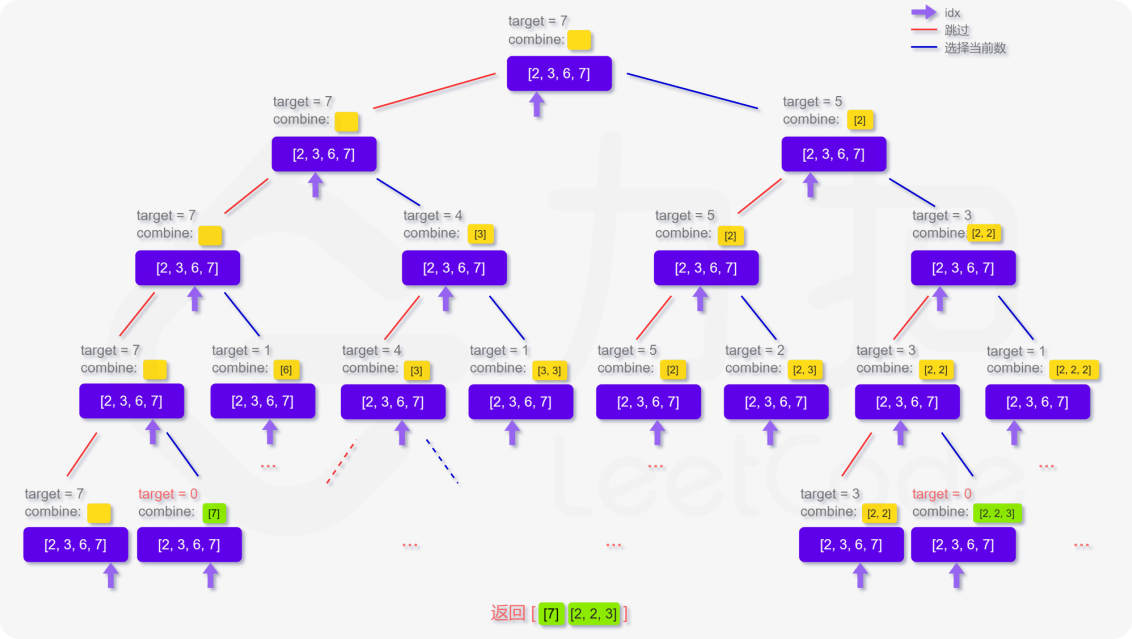
-

39.组合总和

- 一个 **无重复元素** 的整数数组 `candidates` 和一个目标整数 `target` ，找出 `candidates` 中可以使数字和为目标数 `target` 的 所有 **不同组合** ，并以列表形式返回。`candidates` 中的 **同一个** 数字可以 **无限制重复被选取** 。

```
输入: candidates = [2,3,6,7], target = 7
输出: [[2,2,3],[7]]
解释:
2 和 3 可以形成一组候选，2 + 2 + 3 = 7 。注意 2 可以使用多次。
7 也是一个候选， 7 = 7 。
仅有这两种组合。
```

- 思路：**排列组合搜索问题**，采用回溯法求解。



2. 如何处理“数字可重复使用”？

- 在普通的组合问题（如从 n 个数选 k 个，不可重复）中，递归时下一层通常从 `i + 1` 开始。
- 本题特殊性：因为数字可以重复选（例如 `[2, 2, 3]` ），所以在递归调用下一层时，起始索引仍然是 `i` ，而不是 `i + 1` 。
  - 代码体现： `dfs(..., i)` 。这意味着当前数字 `nums[i]` 在这一层被选了之后，下一层依然有机会再次选中它。

3. 如何避免“重复组合”？ （关键点）

- 问题：如果不加控制， `[2, 3]` 和 `[3, 2]` 会被视为两个不同的解，但题目要求它们是一样的。
- 策略：限制搜索方向（顺序性）
  - 我们规定：生成的组合必须是单调非递减的（或者说，一旦决定不再选前面的数，就永远不能再回头选）。
  - 实现方法：引入 `start_index` 参数。
    - 第一层循环从下标 `0` 开始。
    - 如果第一层选了 `nums[0]` （即 2），那么第二层递归时，只能从 `nums[0]` 及之后的数字开始选（ `start_index = 0` ）。
    - 如果第一层选了 `nums[1]` （即 3），那么第二层递归时，只能从 `nums[1]` 及之后的数字开始选（ `start_index = 1` ），绝不允许再回头选 `nums[0]` 。
  - 效果：这样就天然保证了不会出现 `[3, 2]` 这种情况，因为选了 3 之后，2 已经被“屏蔽”在搜索范围之外了。

```
class Solution {
public:
    void dfs(vector<vector<int>>& res,vector<int>& nums,vector<int>& path,int sum,int target,int index){
        // 终止条件
        if(sum==target){ // 路径和为target
            res.push_back(path);
            return;
        }
        if (sum > target) return; // 剪枝
        // 只向后选择数字，不会“回头”选前面的数字
        for(int i=index;i<nums.size();i++){
            // 处理元素
            path.push_back(nums[i]); // 将当前数字加入路径
            // 递归遍历
            // 注意传入i，因为数字允许重复使用，所以下一层仍然可以从当前位置 i 开始选
```

```
            dfs(res,nums,path,sum+nums[i],target,i);
            // 回溯
            path.pop_back();
        }
    }

    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        vector<vector<int>> res; // 存放所有符合条件的组合
        vector<int> path; // 存放当前正在尝试的组合
        int sum = 0; // 当前路径和
        dfs(res,candidates,path,sum,target,0);

        return res;
    }
};
```

-

括号生成

- 数字 `n` 代表生成括号的对数，用于生成所有可能的并且 **有效的** 括号组合。

```
输入: n = 3
输出: ["((()))","(()())","(())()","()(())","()()()"]
```

- 思路：

  我们要生成 n 对括号，意味着最终字符串长度为 $2n$ ，其中包含 n 个 `'('` 和 n 个 `')'` 。
  一个有效的括号序列必须满足两个条件：

  1. 总数限制：左括号和右括号的数量都不能超过 n 。
  2. 匹配限制：在字符串的任意位置，已出现的右括号数量 `right` 绝不能超过已出现的左括号数量 `left` 。（否则就会出现 `())` 这种无法匹配的情况）。

  - 在每一步递归中，面临两个选择：**放左括号** 或 **放右括号**。
    - **添加左括号** `'('`
      - **逻辑**：只要已用的左括号数量 `left` 还没达到上限 `n`，我们就可以放心地加一个左括号。
      - **安全性**：加左括号永远不会导致非法（只要总数不超），因为它总是可以被后续的右括号匹配。
    - **添加右括号** `')'`
      - **逻辑**：只有当 **已用的右括号数量** `right` **少于 已用的左括号数量** `left` 时，才能加右括号。
      - 关键点：
        - 如果 `right == left`，说明目前的左括号都已经有了对应的右括号（或者还没开始），此时再加右括号就会变成 `...)`，导致右括号多于左括号，非法。

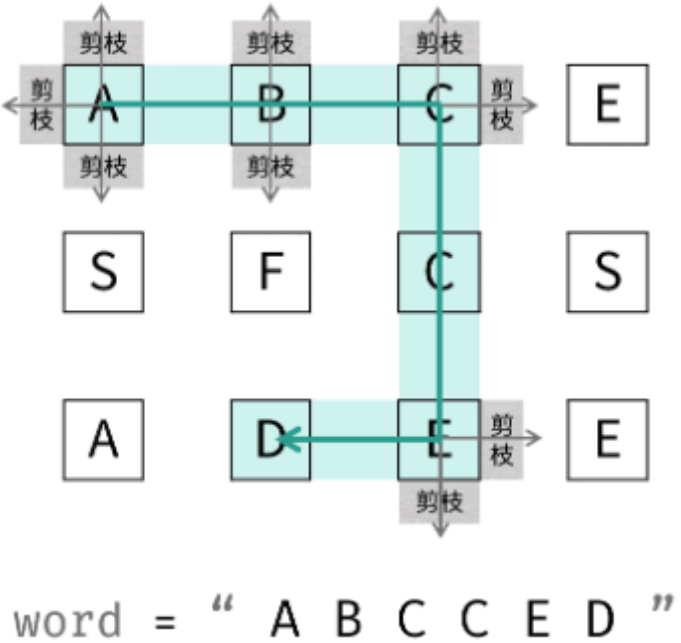
    - 如果 `right > left`，这种情况在之前的步骤就被拦截了，根本不会发生。

```
class Solution {
public:
    void dfs(vector<string>& res,string& path,int n, int left, int right){
        // 终止条件，括号字符串的长度==n * 2
        if(path.size() == n*2){
            res.push_back(path);
            return;
        }
        // 添加左括号 '('
        // 条件：左括号数量还没用完（小于 n）
        if (left < n) {
            path.push_back('(');            // 做选择
            dfs(res, path, n, left + 1, right); // 递归：左括号数+1
            path.pop_back();                // 撤销选择（回溯）
        }

        // 添加右括号 ')'
        // 条件：右括号数量少于左括号数量（保证有效性，不会出现 "())" 这种情况）
        if (right < left) {
            path.push_back(')');            // 做选择
            dfs(res, path, n, left, right + 1); // 递归：右括号数+1
            path.pop_back();                // 撤销选择（回溯）
        }




    }

    vector<string> generateParenthesis(int n) {
        vector<string> res;
        string path;
        dfs(res,path,n,0, 0);
        return res;
    }
};
```

-

单词搜索

- 一个 `m x n` 二维字符网格 `board` 和一个字符串单词 `word` 。判断word是否在board中。单词必须按照字母顺序，通过相邻的单元格内的字母构成。




```
输入: board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word = "ABCCED"
输出: true
```

- 思路:

```
- 递归参数: 当前元素在矩阵 board 中的行列索引 i 和 j ，当前目标字符在 word 中的索引 k 。
- 终止条件:
  1. 返回 false ： (1) 行或列索引越界 或 (2) 当前矩阵元素与目标字符不同 或 (3) 当前矩阵元素已访问过 （ (3) 可合并至 (2) ） 。
  2. 返回 true ： k = len(word) - 1 ，即字符串 word 已全部匹配。
- 递推工作:
  1. 标记当前矩阵元素: 将 board[i][j] 修改为 空字符 '' ，代表此元素已访问过，防止之后搜索时重复访问。
  2. 搜索下一单元格: 朝当前元素的 上、下、左、右 四个方向开启下层递归，使用 或 连接（代表只需找到一条可行路径就直接返回，不再做后续 DFS ），并记录结果至 res 。
  3. 还原当前矩阵元素: 将 board[i][j] 元素还原至初始值，即 word[k] 。
- 返回值: 返回布尔量 res ，代表是否搜索到目标字符串。
```

```cpp
class Solution {
    int rows,cols;
public:
    bool exist(vector<vector<char>>& board, string word) {
        rows = board.size();
        cols = board[0].size();
        // 对每个元素进行dfs(board上每个位置元素都有可能是单词的起点)
        for(int i=0; i<rows; i++){
            for(int j=0; j<cols;j++){
                if(dfs(board,word,i,j,0)) return true;
            }
        }
        return false;
    }
```

```
    bool dfs( vector<vector<char>>& board,string& word,int i,int j,int k){
        // 终止条件
        if( i>=rows||i<0||j>=cols||j<0||board[i][j]!=word[k]) { // 越界、字母不
一致
            return false;
        }
        if( k==word.size()-1 ) {
            return true; // 长度为word长度（字母均一致）
        }
        // 元素处理
        board[i][j]='\0'; // 将遍历过的字母设置为空，代表已访问
        // 递归遍历，上下左右四个方向
        bool res = dfs(board,word,i+1,j,k+1)||
                   dfs(board,word,i-1,j,k+1)||
                   dfs(board,word,i,j+1,k+1)||
                   dfs(board,word,i,j-1,k+1);
        board[i][j] = word[k]; //  回溯，将 board[i][j] 元素还原至初始值

        return res;
    }
};
```

-

131.分割回文串

- 将 `s` 分割成一些 子串，使每个子串都是 **回文串** 。返回所有可能的分割方案。

```
输入: s = "aab"
输出: [["a","a","b"],["aa","b"]]
```

- 思路：

我们可以把这个问题想象成在走一棵**决策树**：

- 树的每一层：代表字符串中的一个切割起始位置 `index` 。
- 树的每一个分支：代表从当前 `index` 开始，向后截取不同长度的子串（例如截取长度 1、长度 2、长度 3...）。
- 剪枝（Pruning）：如果截取的子串不是回文串，那么这个分支就是死路，直接放弃，不再继续向下递归。

- 终止条件：
  如果 `index` 到达了字符串的末尾（`index == s.size()`），说明整个字符串已经被成功分割成了若干个回文子串。此时将当前的 `path` 加入结果集 `res`，并返回。
- 选择过程（遍历切割点）：
  从 `index` 开始，枚举结束位置 `i`（从 `index` 到 `s.size()-1`）：
  1. 截取子串：获取子串 `sub = s[index...i]`。
  2. 判断回文：检查 `sub` 是否是回文串。
     - 如果是回文：
       - 做选择：将 `sub` 加入 `path`。
       - 递归：调用 `dfs(i + 1)`，去处理剩余的字符串部分。
       - 撤销选择（回溯）：递归返回后，将 `sub` 从 `path` 中移除，以便尝试下一个长度的子串（即让 `i` 增加，截取更长的子串）。
     - 如果不是回文：
       - 剪枝：直接跳过，不进入递归。因为如果第一段就不是回文，后面无论怎么切，整个方案都是无效的。

```
class Solution {
public:
    bool judge(string s){
        if( s.size()==0||s.size()==1 ) return true;
        for( int j=s.size()-1,i=0;i<=j;j--,i++ ){
            if (s[j]!=s[i]){
                return false;
            }
        }
        return true;
    }

    void dfs(vector<vector<string>>& res,vector<string>& path,string s,int
index){
        // 终止条件
        if(index==s.size()){
            res.push_back(path);
            return;
        }
        // 从 index 开始，截取不同长度的子串 s[index...i]
        for(int i=index;i<s.size();i++){
            string t = s.substr(index,i-index+1); // 获取子串
            if(judge(t)){ // 剪枝：只有当子串是回文时，才继续递归
                // 处理元素
                path.push_back(t); // 将当前回文子串添加进路径
                // 递归处理
                dfs(res,path,s,i+1);
                // 回溯，撤销选择
                path.pop_back();
            }
        }
    }

    vector<vector<string>> partition(string s) {
        vector<vector<string>> res;
        vector<string> path;
        dfs(res,path,s,0);

        return res;
```

```
    }
};
```

-